Create a new file at lib/examples/structs/conversation.ex and add the following code:

elixir_examples/lib/examples/structs/conversation.ex

```
defmodule Examples.Structs.Conversation do
  defstruct [:other_party, messages: []]

  def new(sms_list) do
    other_party = List.first(sms_list) |> Examples.Structs.Sms.other_party()
    %__MODULE__{other_party: other_party, messages: sms_list}
  end

  def append(conversation, sms) do
    new_messages = conversation.messages ++ [sms]
    Map.put(conversation, :messages, new_messages)
  end

  def clear(conversation) do
    Map.put(conversation, :messages, [])
  end
end
```

And try it out:

```
$ iex -S mix
iex> alias Examples.Structs.{Conversation, Sms}
iex> from_me = %Sms{from: "me", to: "you", status: "delivered"}
iex> from_you = %Sms{from: "you", to: "me", status: "received"}
iex> convo = Conversation.new([from_me])

%Examples.Structs.Conversation{
  messages: [
    %Examples.Structs.Sms{
      body: nil, from: "me", status: "delivered", to: "you"
    }
  ],
  other_party: "you"
}

iex> convo = Conversation.append(convo, from_you)

%Examples.Structs.Conversation{
  messages: [
    %Examples.Structs.Sms{
      body: nil, from: "me", status: "delivered", to: "you"
    },
    %Examples.Structs.Sms{
      body: nil, from: "you", status: "received", to: "me"
    }
  ],
  other_party: "you"
}
```

This example combines the things you've learned so far in a way that you'll likely see in an Elixir application. The one new concept is the __MODULE__